

Troubleshooting Guide

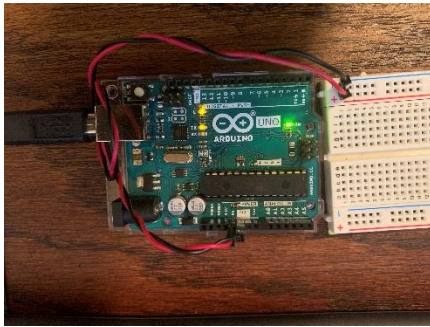
1. Using the serial monitor to track your code

As your codes get more and more complex, its important to be able to track what your code is doing as it runs, a simple trick it to “print” these lines of code onto the serial monitor to gain a visual understanding on what your Arduino is doing.

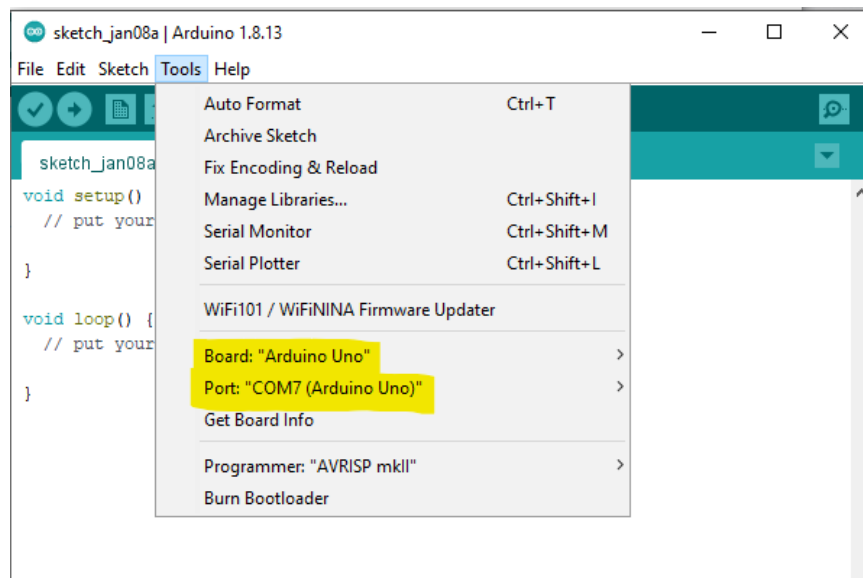
How to setup your serial monitor.

Step 1:

- Ensure the Arduino is plugged into the computer and is powered on (a green light will illuminate when powered on)



- Set the board type to “Arduino Uno” and port to the corresponding port number the Arduino is plugged into. (ex. COM7) This will typically say “(Arduino Uno)” next to the correct port.

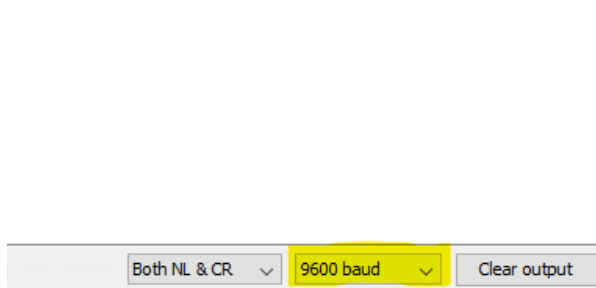


Step 2:

- Setup serial communications. To do this, you must add a line of code. Inside the `void setup()` function, add the line `Serial.begin(9600);` to setup the communications with the serial monitor.

```
void setup() {  
  
    Serial.begin(9600);  
}
```

- To open the serial monitor, click on Tools (at the top) → Serial Monitor, or press Ctrl+Shift+M to pull up the serial monitor. Make sure that the number at the bottom of the serial monitor matches the number set after the command `Serial.begin(9600);`



Step 3:

- Now you are ready to start tracking your code using the serial monitor. By using the code `Serial.println();` inside your `void loop()` function, you can have whatever you want appear on the monitor in real time as your code progresses.

Example:

- Say for example you have this code:

```
void setup() {  
  
    Serial.begin(9600);  
}  
  
void loop() {  
  
    int Switch = 0;  
    delay(1000);  
    Switch = 1;  
    delay(1000);  
}
```

- This is a simple code that for every 1 second, the variable “Switch” goes from 0 to 1. Now if we want to see in real time, how the Arduino is toggling the variable, we can add `Serial.println();` command after the variable changes to visually see how the code is performing.

```

void setup() {

    Serial.begin(9600);

}

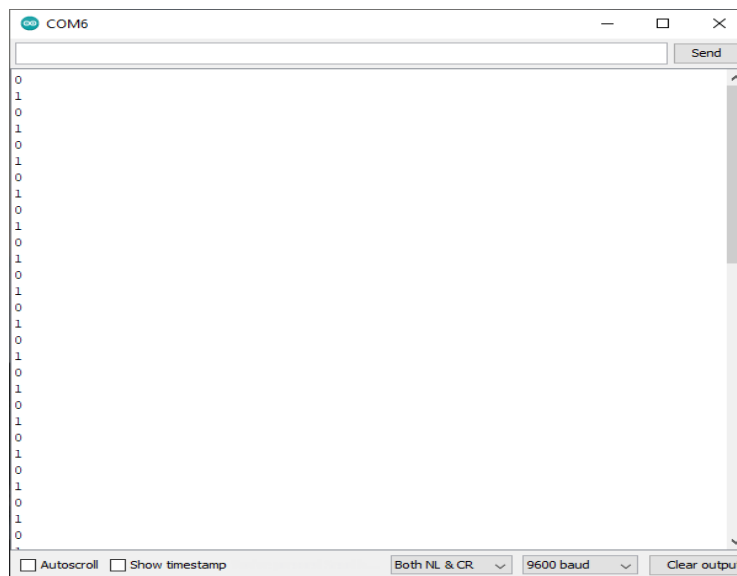
void loop() {

    int Switch = 0;
    Serial.println(Switch);
    delay(1000);
    Switch = 1;
    Serial.println(Switch);
    delay(1000);

}

```

- Now if we take a look at the serial monitor, we will notice 0's and 1's appearing every 1 second.



- we can even go a little further and define what these 0's and 1's represent. This can help keep better track of what is actually going on in the code.

```

void setup() {

    Serial.begin(9600);

}

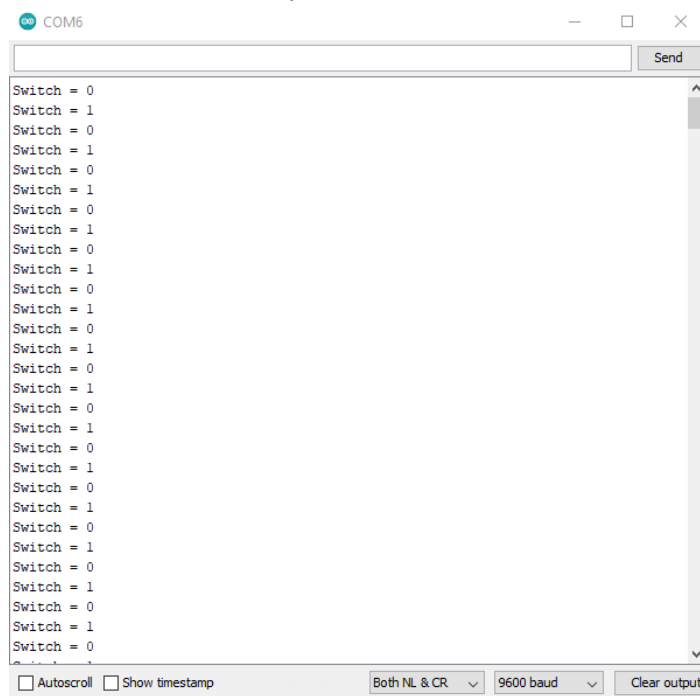
void loop() {

    int Switch = 0;
    Serial.print("Switch = ");
    Serial.println(Switch);
    delay(1000);
    Switch = 1;
    Serial.print("Switch = ");
    Serial.println(Switch);
    delay(1000);

}

```

- By adding more `Serial.print();` commands we are now telling the serial monitor to print out a string “Switch =” before every Switch change. It is important to know that `Serial.print();` will print everything on the same line until you run a `Serial.println();` command. You always put the statement you want to appear at the end of a line inside a `Serial.println();` command. Shown below is the output of the above code onto the serial monitor.



- This “trick” can be used to keep track of anything that you might be unsure of as your code progresses. For example, you can put anything inside a `Serial.println();` command, such as a variable, string, array, Boolean, even **analog** and **digital** readings.

2. Using a multimeter:

1. Continuity check:

- a. Make sure your circuit is unplugged from any power supply (batteries, USB port, etc)
- b. Use the continuity checking feature symbolized as indicated with a yellow circle in the image below.



- c. This technique can be used to check soldered joints for any cracks or breadboard connections that may be loose. The beep sound produced by the meter indicated a connection. Note that this test is only valid for wires connection and not electrical components such as resistors and capacitors.
- d. Note that this position on the meter is also designed to measure resistance in case you needed to validate a resistor value.

2. Voltage level:

- a. Voltage level indicated on the screen should reflect were current is flowing to and from.
- b. Simply switch to the DC voltage option on the meter indicated in yellow below. When on voltage reading the polarity of your probe does not matter, the meter will indicate flipped polarity with a negative sign.



3. Commonly Encountered Errors

Various errors can occur before your code successfully uploads to your Arduino UNO; however, simple troubleshooting will solve most of those that you will encounter.



```
File Edit Sketch Tools Help
exercise_4
int switchState = 0;

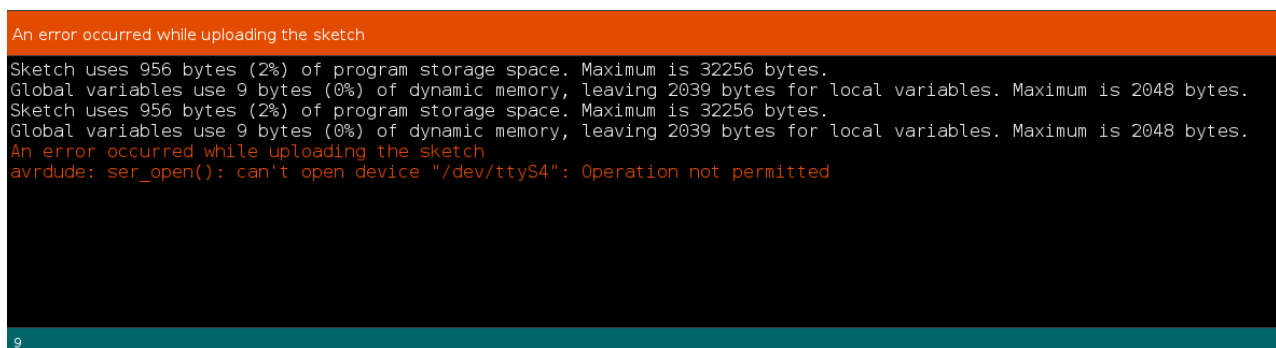
void setup() {
  // put your setup code here, to run once:
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(2, INPUT)
}
```

expected ';' before '}' token

/home/john/Documents/Mechatronics_2021/lab_1/exercise_4/exercise_4.ino: In function 'void setup()':
exercise_4:10:1: error: expected ';' before '}' token
}
^
exit status 1
expected ';' before '}' token

- Above is a prevalent error which many students in the past have sent screenshots of asking for help.
- This simply means you are missing a semi-colon at the ends of one of your lines of code, meaning this is one of the easiest errors to fix, since you would simply type `pinMode(2, INPUT);`

Another common error encountered quite often is pictured below utilizing the same code from above with the added semicolon.



```
An error occurred while uploading the sketch
Sketch uses 956 bytes (2%) of program storage space. Maximum is 32256 bytes.
Global variables use 9 bytes (0%) of dynamic memory, leaving 2039 bytes for local variables. Maximum is 2048 bytes.
Sketch uses 956 bytes (2%) of program storage space. Maximum is 32256 bytes.
Global variables use 9 bytes (0%) of dynamic memory, leaving 2039 bytes for local variables. Maximum is 2048 bytes.
An error occurred while uploading the sketch
avrdude: ser_open(): can't open device "/dev/ttyS4": Operation not permitted
```

- This means that you have not selected the port you want to upload your code too, as described on page 1 of this document.

- In order to fix this, you simply go into the tools tab at the top of the Arduino IDE and select the port which shows a connection to the Arduino UNO.

These two errors have been seen by TAs countless times in past semesters, with other more difficult to solve errors appearing fairly few and far between. If throughout your time in this lab, you do encounter other errors, feel free to reach out to your TA and ask for help, however, you should do some research on your own prior to asking for help.